ECE 590
Neural-Network-Based Large Language Models

Tuesdays and Thursdays 1:25-2:40
Fitzpatrick Schiciano A 1464

ECE 590
Neural-Network-Based Large Language Models

Lecture 3
01/15/2026

Why the Logistic Link Function?

- Fundamental to logistic regression is the logistic or sigmoid link function

$$z_i = \boldsymbol{w}^T \boldsymbol{x}_i + b$$

$$p(Y_i = 1 | X_i = \boldsymbol{x}_i) = \frac{\exp(z_i)}{1 + \exp(z_i)} = \sigma(z_i)$$

- Why the sigmoidal link function, $\sigma(z)$?

Why the Logistic Link Function?

- Assume that $p(Y, X)$ is the true underlying distribution of the data, where Y is a binary random variable, $Y \in \{0,1\}$ and X is the *d*-dimensional data vector $X \in R^d$

- (X, Y) are joint random variables, the distribution of which we seek to model

- From the laws of probability

$$p(Y|X) = \frac{p(Y, X)}{p(X)} = \frac{p(Y, X)}{p(Y = 0, X) + p(Y = 1, X)}$$

Why the Logistic Link Function?

$$p(Y=1|X) = \frac{p(Y=1,X)}{p(X)} = \frac{p(Y=1,X)}{p(Y=0,X)+p(Y=1,X)}$$

- We therefore have

$$p(Y=1|X) = \frac{p(Y=1,X)/p(Y=0,X)}{1+p(Y=1,X)/p(Y=0,X)}$$

Likelihood Ratio

$$p(Y=1|X) = \frac{p(Y=1,X)/p(Y=0,X)}{1+p(Y=1,X)/p(Y=0,X)}$$

- The likelihood ratio is defined as

$$L(X) = \frac{p(Y=1,X)}{p(Y=0,X)} = \frac{p(Y=1|X)p(X)}{p(Y=0|X)p(X)} = \frac{p(Y=1|X)}{p(Y=0|X)}$$

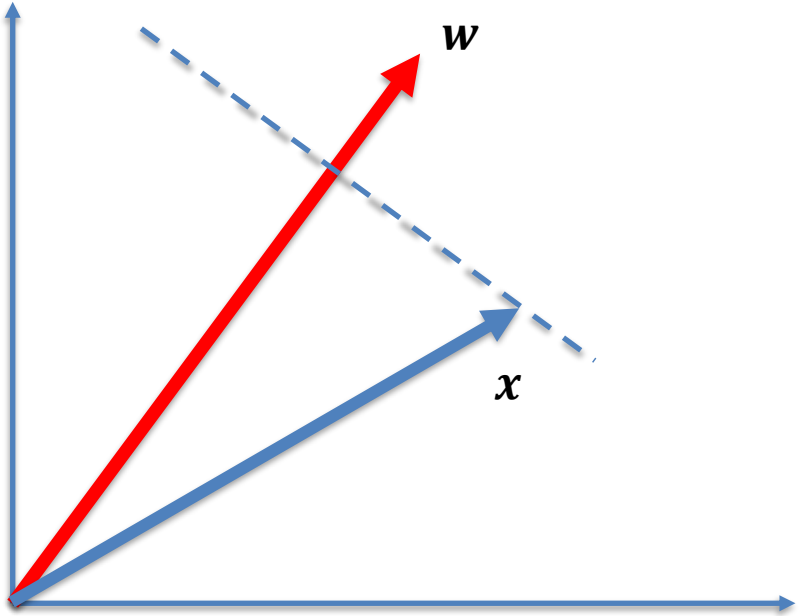
- Note that $L(X)$ is always positive, and it represents the relative likelihood of data X coming from class $Y=1$ vs class $Y=0$

Modeling the Likelihood Ratio

$$p(Y = 1|X) = \frac{L(X)}{1 + L(X)} \qquad p(Y = 0|X) = 1 - p(Y = 1|X)$$

$$L(X) = \frac{p(Y = 1|X)}{p(Y = 0|X)}$$

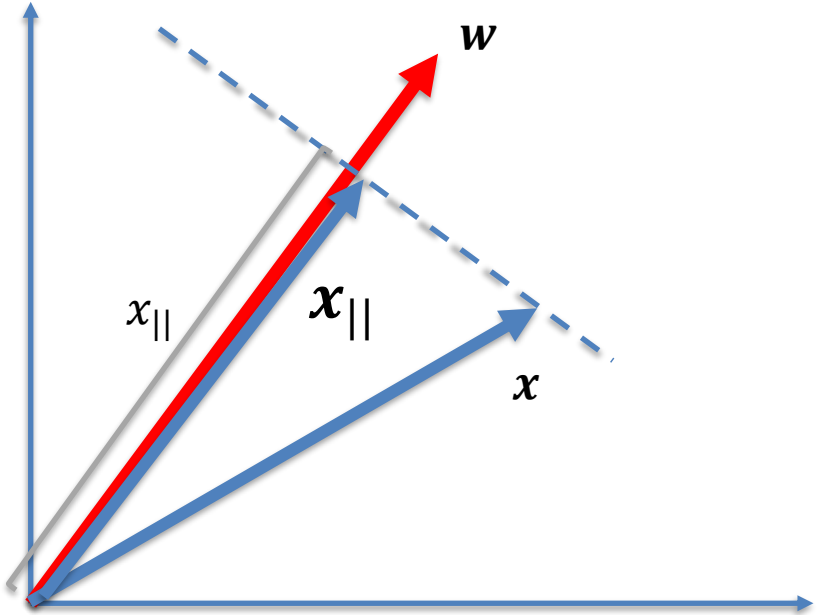
- Much of machine learning concerns modeling the likelihood $L(X)$
- Simple logistic regression is based on the model $L(X) = \exp[\boldsymbol{w}^T X + b]$
- This is a simple linear model: Linear decision surfaces perpendicular to vector w in $\mathbb{R}^d$



All x on a *hyperplane* perpendicular to w yield the same predicted probability for $p(Y = 1|X = x)$

$$p(Y = 1|X) = \frac{L(X)}{1 + L(X)}$$

$$L(X) = \exp[\boldsymbol{w}^T X + b]$$

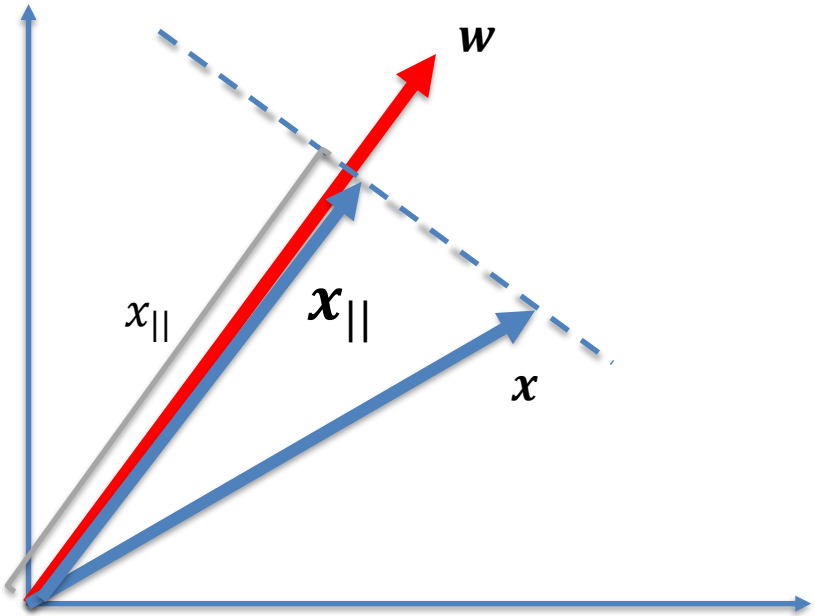


- Along the line specified by w the model of $L(X)$ is a generalized linear model

$$L(x) = \exp[\|\boldsymbol{w}\|_2 x_{||} + b]$$

- Slope$\|\boldsymbol{w}\|_2$ and intercept b

Template Matching



- Vector $\boldsymbol{w}$ defines a template for class $Y = 1$, and we perform inner-product-based matching to specify the probability of that class

$$L(x) = \exp[\|\boldsymbol{w}\|_2 x_{||} + b]$$

- Assumption of linear model in space of data X very limiting

Template Matching in Feature Space

$$p(Y = 1|X) = \frac{L(X)}{1 + L(X)} \qquad p(Y = 0|X) = 1 - p(Y = 1|X)$$

$$L(X) = \frac{p(Y = 1|X)}{p(Y = 0|X)}$$

- Move beyond linear model in the space of the data X

- Learn a nonlinear transformation $X \to \psi(X)$ where $\psi(X)$ is a learned <u>nonlinear</u> transformation of the data

- Will often model $\psi(X)$ with a neural network (to be explained in detail later)

Template Matching in Feature Space

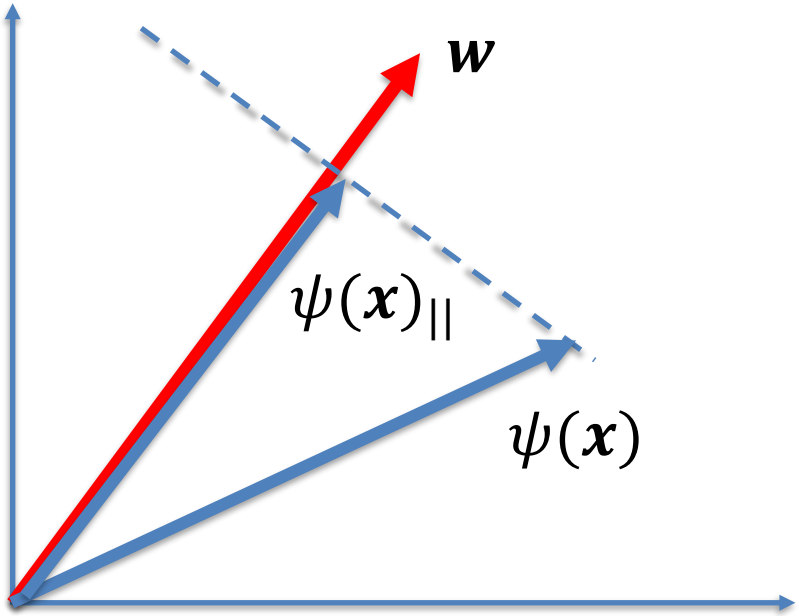
$$p(Y = 1|X) = \frac{L(X)}{1 + L(X)} \qquad p(Y = 0|X) = 1 - p(Y = 1|X)$$

$$L(X) = \frac{p(Y = 1|X)}{p(Y = 0|X)}$$

- Model likelihood ratio as $L(X = \boldsymbol{x}) = \exp[\boldsymbol{w}^T \psi(x) + b]$

- The weight vector $\boldsymbol{w}$ is in the space of features $\psi(\boldsymbol{x})$ and the dimension of these features (dimension of vector $\boldsymbol{w}$) is typically different from the dimension of $\boldsymbol{x}$

Template Matching in Feature Space

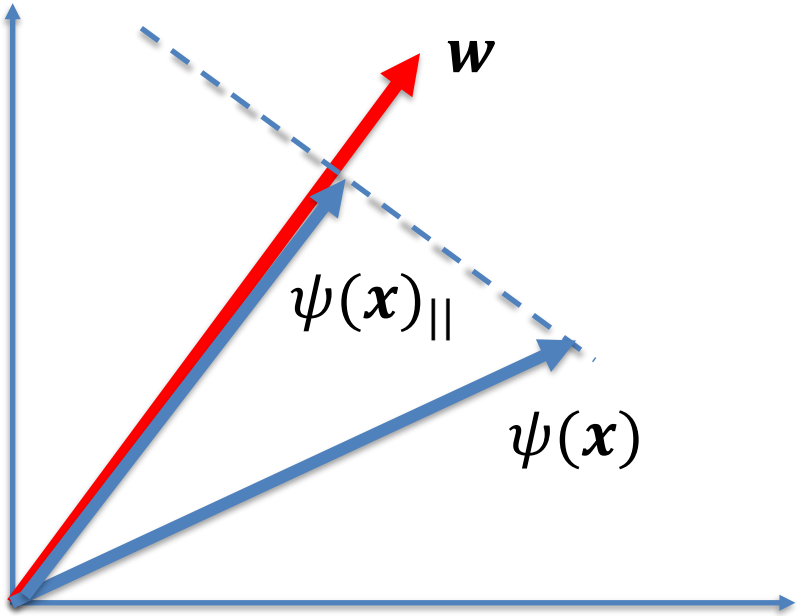


- Vector w defines a template for class $Y = 1$, and we perform inner-product-based matching in feature space $\psi(\boldsymbol{x})$

$$L(\boldsymbol{x}) = \exp[\|\boldsymbol{w}\|_2 \psi(\boldsymbol{x})_{||} + b]$$

- Because the learned function $\psi(\boldsymbol{x})$ is nonlinear, the model $\|\boldsymbol{w}\|_2\ \psi(\boldsymbol{x})_{||} + b$ is a nonlinear function of the data x

The Heart of Neural Networks & Deep Learning



- We retain the idea of generalized linear models

$$L(\boldsymbol{x}) = \exp[\|\boldsymbol{w}\|_2 \psi(\boldsymbol{x})_{||} + b]$$

- However, rather than building the linear model in the data space $\boldsymbol{x}$ we build the linear model in a learned feature space $\psi(\boldsymbol{x})$

- Multi-layered ("deep") neural networks will be used to model $\psi(\boldsymbol{x})$

Generalization of Nonlinearity

- When considering binary classification problems, at the output layer of the model we will generally, continue to use the logistic link function, with

$$p(Y = 1|X = \boldsymbol{x}) = \sigma[\boldsymbol{w}^T \psi(\boldsymbol{x}) + b]$$

- A (nonlinear) neural network will be used to manifest the function $\psi(\boldsymbol{x})$, and the overall model is nonlinear

- Fundamental neural network paradigm: Learn a *nonlinear* transformation $\psi(\boldsymbol{x})$ to feature space, and then employ a *linear* model in that feature space

- This construct will be employed from multiple perspectives in large language models

Generalization of Nonlinearity

- The use of the logistic link function, $\sigma(\cdot)$, was guided by a binary classification setup

- We will generalize this concept to the multi-class case (multiple words in language) via use of the softmax link function, for which the logistic link is a special case

- Within neural networks, we will repeatedly implement functions of the form

$$f[\boldsymbol{W}\boldsymbol{z} + b]$$

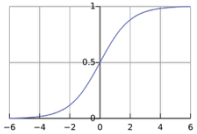
where now $\boldsymbol{z}$ are intermediate features within the neural network used to ultimately manifest the nonlinear transformation $\psi(\boldsymbol{x})$ and $f(\cdot)$ is a *pointwise* activation function (nonlinearity)

- Embedded within the neural network, there is no longer an explicit connection to binary classification, and we will generalize the form of the nonlinearity $f(\cdot)$

Generalization of Nonlinearity

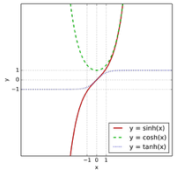
- We will discuss this in more detail later in the semester, but a brief discussion here

- Historically, based on logistic regression and insights from mammalian neurons, the following nonlinearities were used often



$$f(\alpha) = \sigma(\alpha) = \frac{\exp(\alpha)}{1 + \exp(\alpha)}$$

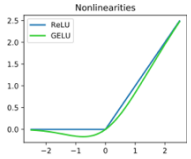
Logistic function



$$f(\alpha) = \tanh(\alpha) = \frac{\sinh(\alpha)}{\cosh(\alpha)} = \frac{\exp(\alpha) - \exp(-\alpha)}{\exp(\alpha) + \exp(-\alpha)}$$

Hyperbolic Tangent function

- More recent interest in using



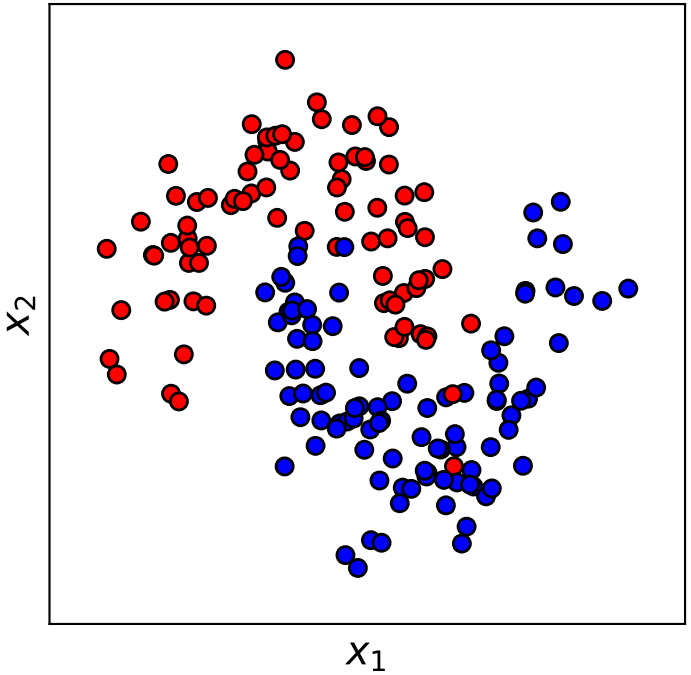
$$f(\alpha) = \mathrm{ReLU}(\alpha) = \begin{cases} \alpha, & \text{if } \alpha \geq 0 \\ 0, & \text{if } \alpha < 0 \end{cases}$$

Rectified linear Unit function

Images from Wikipedia

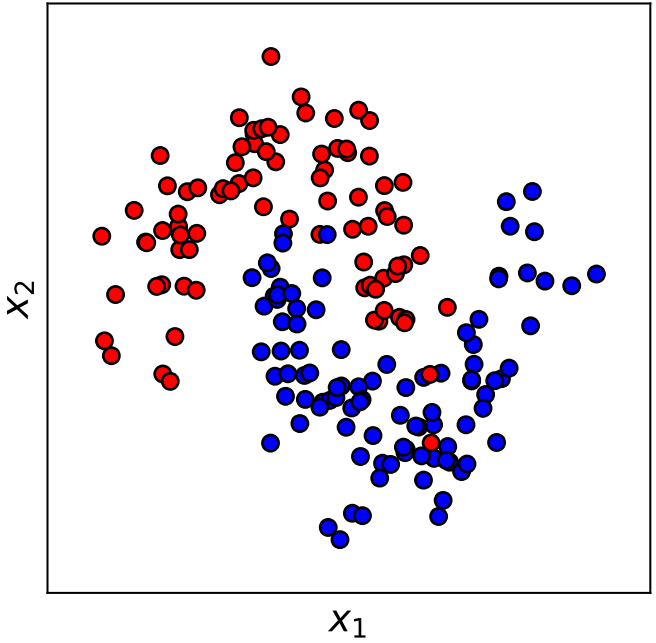
Logistic Regression In Data Space

- Also referred to as a generalized linear model
- Can only split data by linear trends



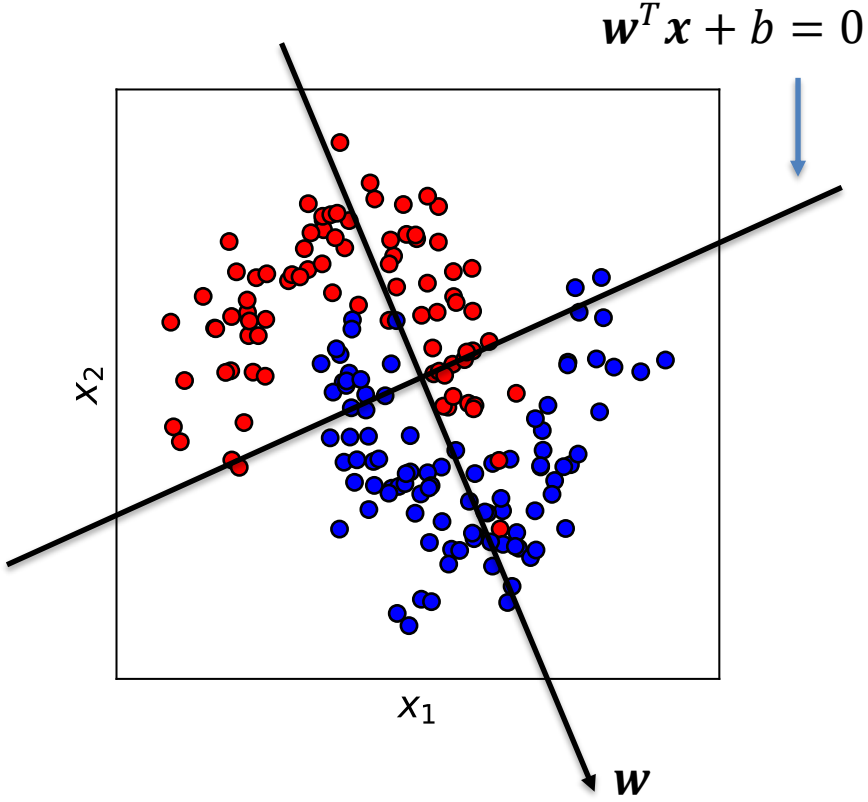
Decision Surface of Logistic Regression

- Linear classifiers can only represent limited relationships
- Often want to use a classifier that can handle non-linearities.
- Many different ways of creating non-linear classifiers



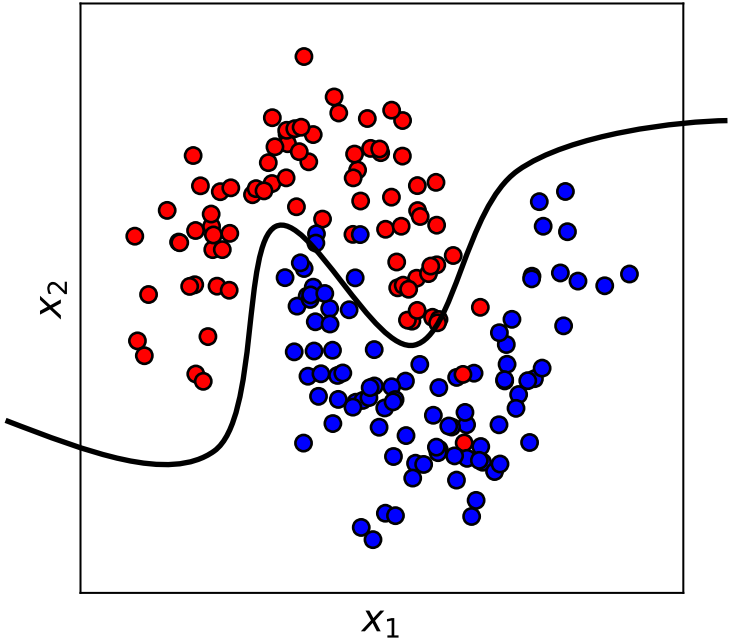
Decision Surface of Logistic Regression

- Linear classifiers can only represent limited relationships
- Often want to use a classifier that can handle non-linearities.
- Many different ways of creating non-linear classifiers



Decision Surface of Logistic Regression

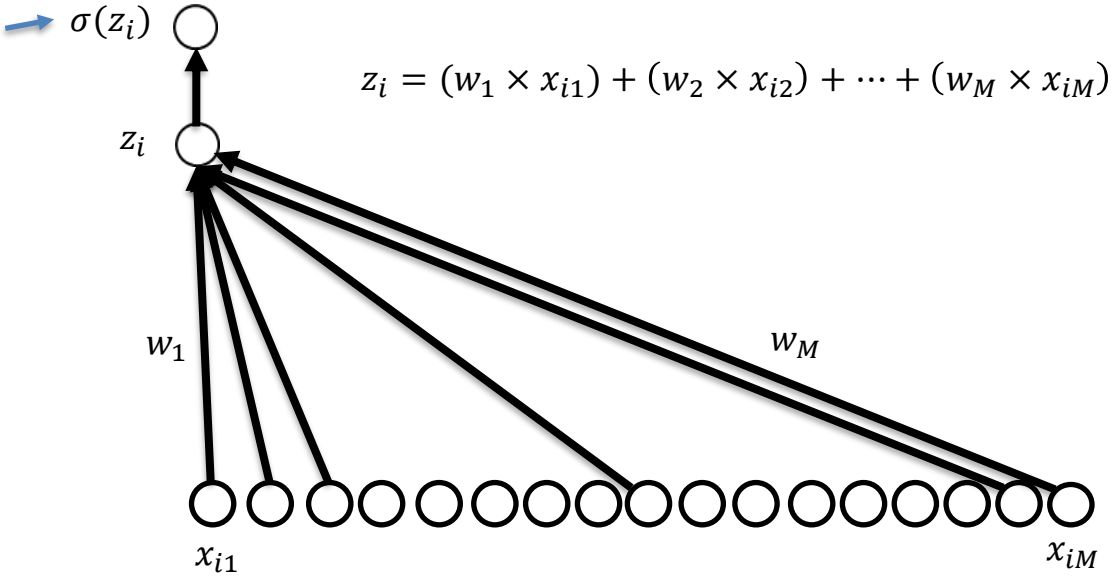
- Linear classifiers can only represent limited relationships
- Often want to use a classifier that can handle non-linearities.
- Many different ways of creating non-linear classifiers



What is then $\boldsymbol{w}$?

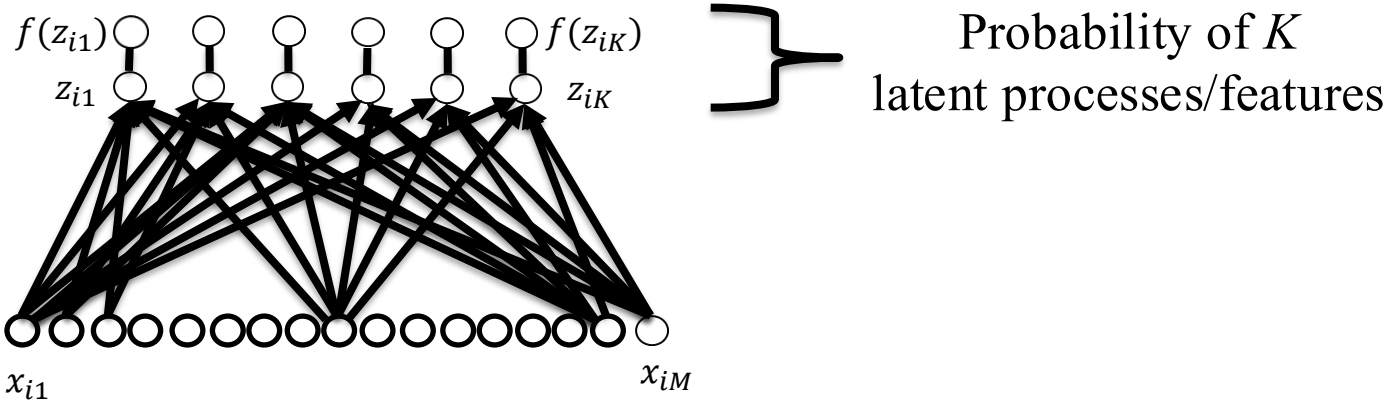
Generalize the Idea of Logistic Regression

Probability of a single outcome

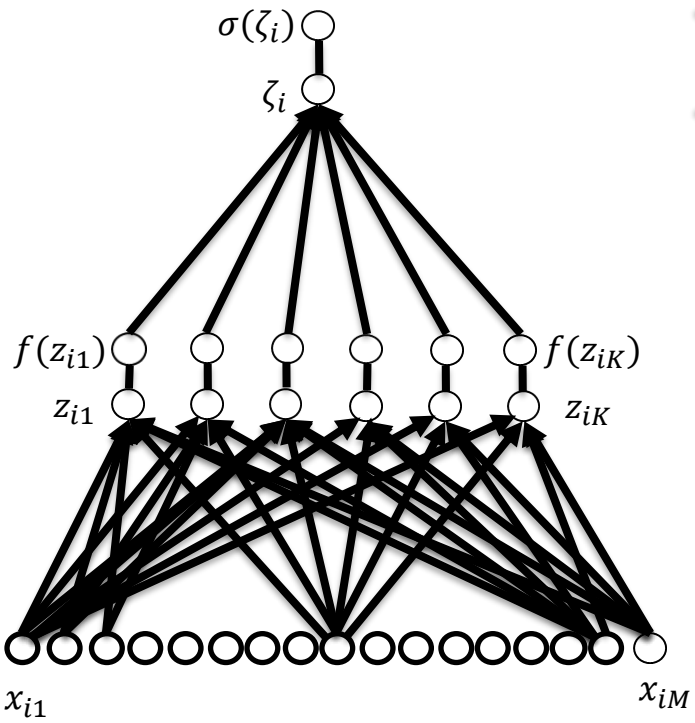


$$z_i = (w_1 \times x_{i1}) + (w_2 \times x_{i2}) + \cdots + (w_M \times x_{iM}) + b$$

Generalization of Logistic Regression: Learned Features



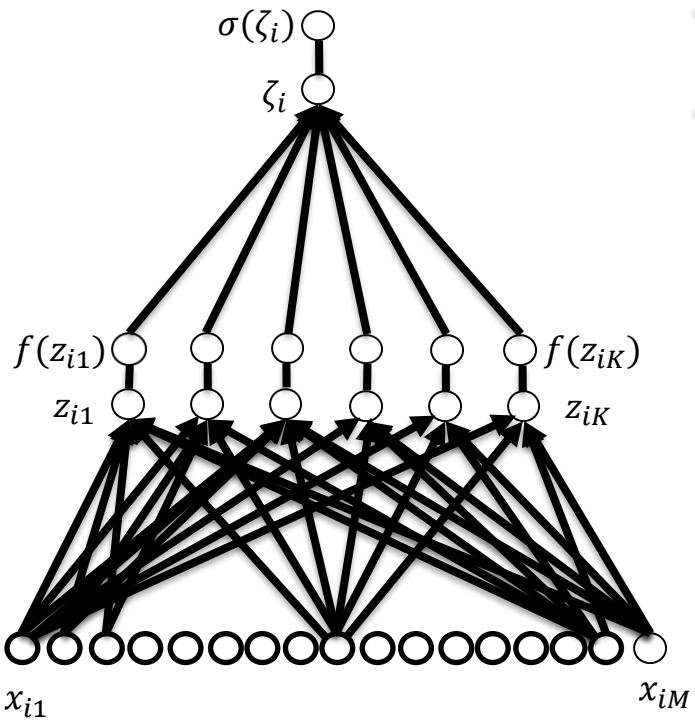
Extended Logistic Regression



Probability of particular outcome

Probability of K latent processes

Extended Logistic Regression



Probability of particular outcome

Probability of K latent processes

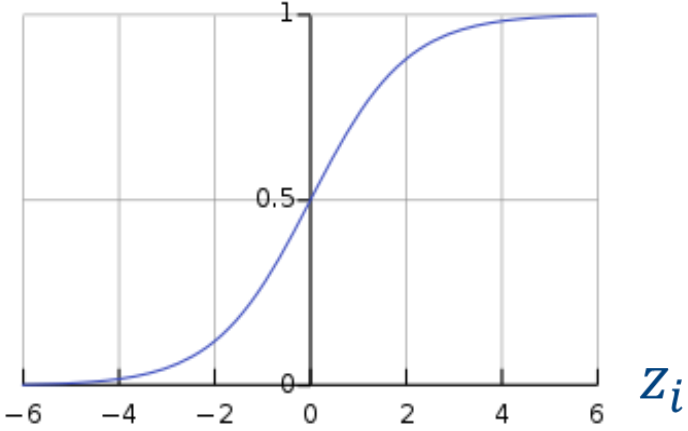
May be viewed as logistic regression on K latent features, rather than directly on the M components of raw data

Recall Logistic Regression

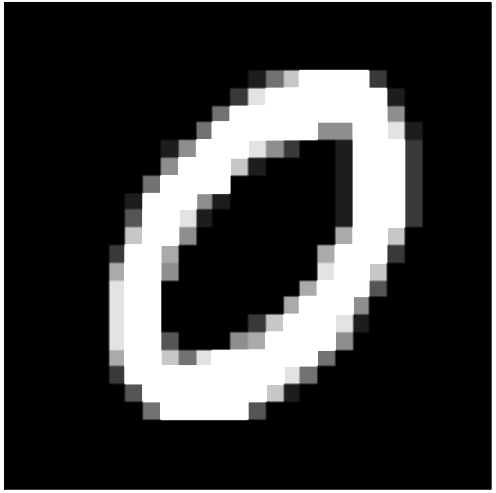
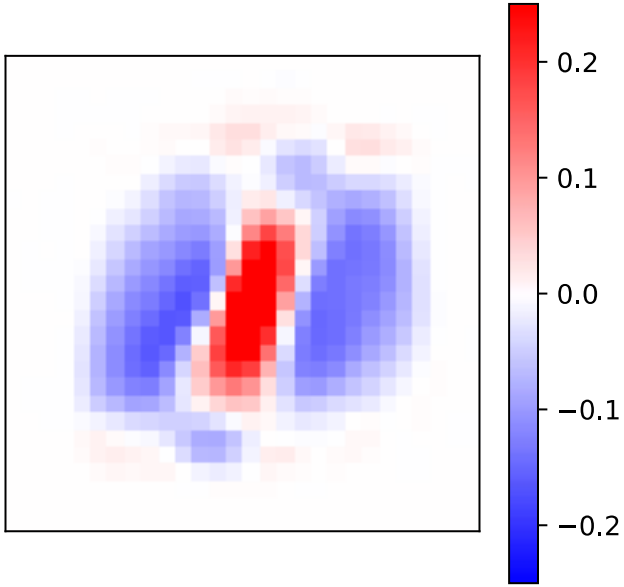
$$z_i = b \quad + (w_1 \times x_{i1}) + (w_2 \times x_{i2}) + \cdots + (w_M \times x_{iM})$$

$$= b \quad + x_i \cdot w$$

$$p(y_i = 1 | x_i) = \sigma(z_i)$$

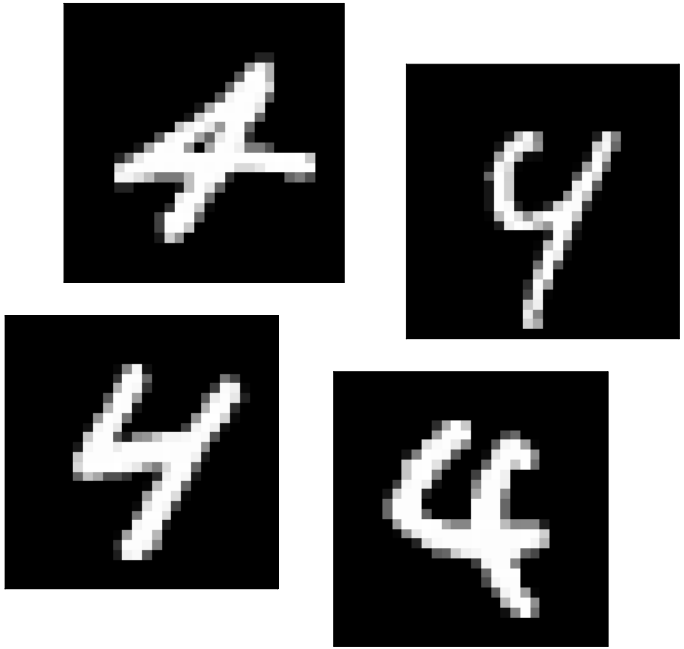


Why Limit Ourselves to Only One Filter?

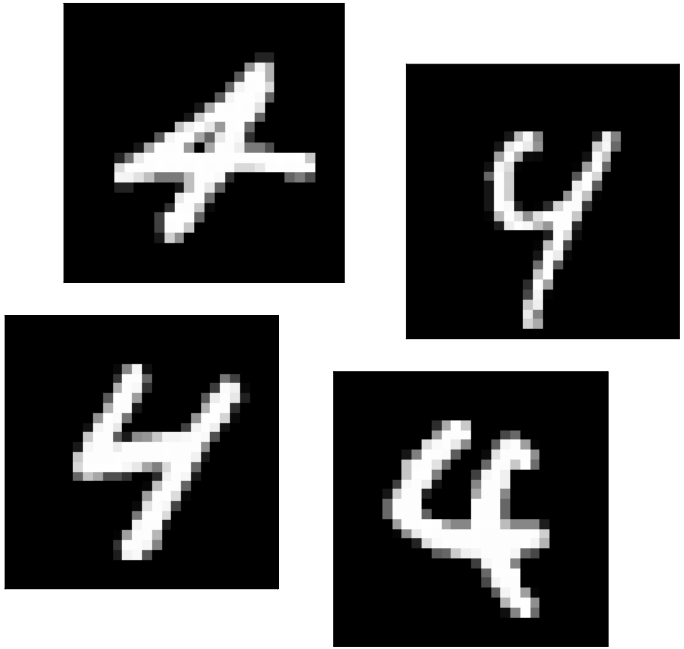
$\sigma($  $\times$ 

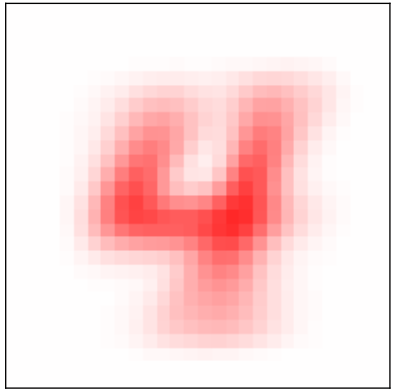
$)$

Return to MNIST: Many ways of writing “4”



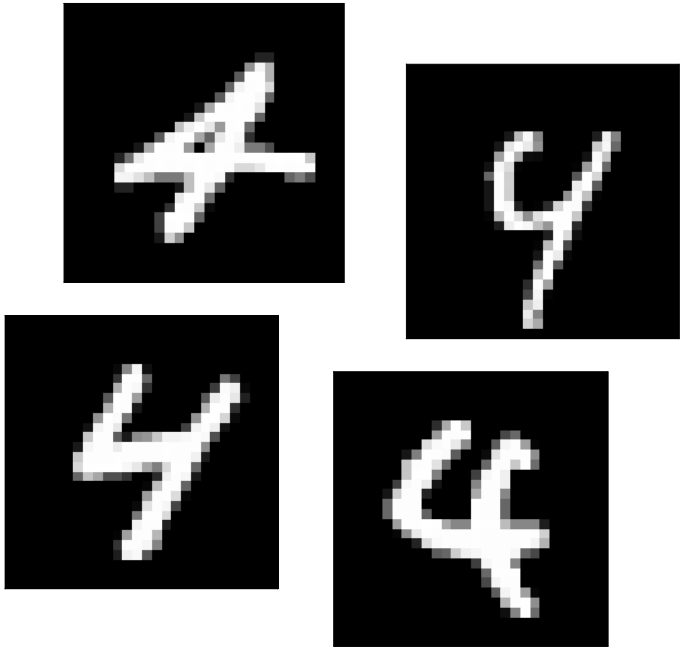
Return to MNIST: Many ways of writing “4”

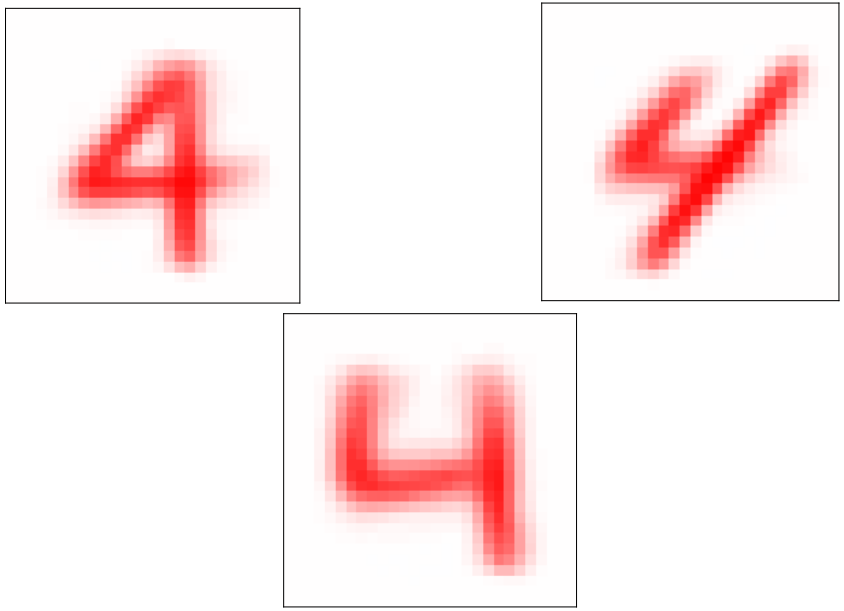




Single Filter (*e.g.*, Logistic Regression/ “Shallow Learning”) only uses one filter, looks for the average shape

Return to MNIST: Many ways of writing “4”





Multiple filters can look for *subtypes* indicative of different ways of writing “4”

Why Limit Ourselves to Only One Filter?

Introduce *K* Filters

$$\left.\begin{aligned} z_{i1} &= b_1 + x_i \cdot w_1 \\ z_{i2} &= b_2 + x_i \cdot w_2 \\ &\vdots \\ z_{iK} &= b_K + x_i \cdot w_K \end{aligned}\right\}$$

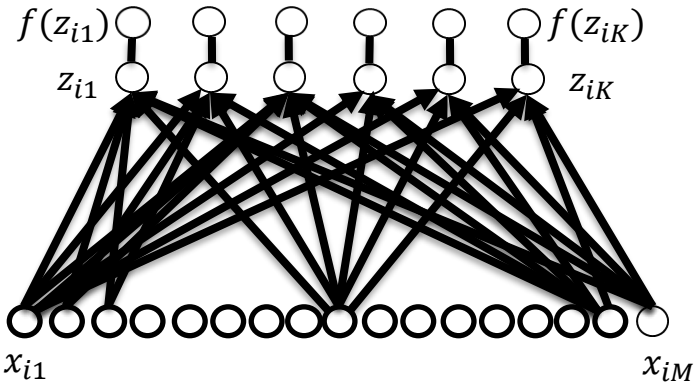
Project data x_i onto *K* filters: $w_1, \ldots, w_K$

Why Limit Ourselves to Only One Filter?

Introduce *K* Filters

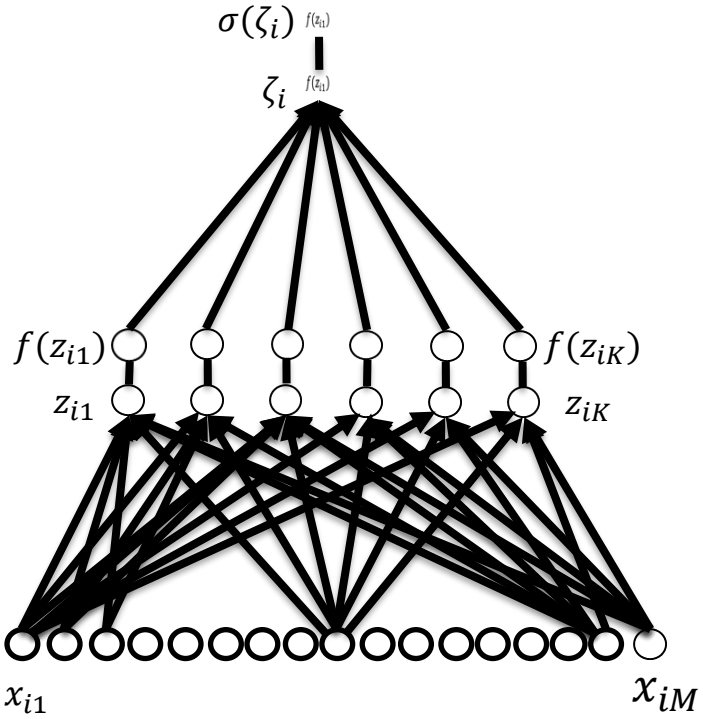
$$z_{i1} = b_1 + x_i \cdot w_1$$
$$z_{i2} = b_2 + x_i \cdot w_2$$
$$\vdots$$
$$z_{iK} = b_K + x_i \cdot w_K$$

Project data x_i onto *K* filters: $w_1, \ldots, w_K$

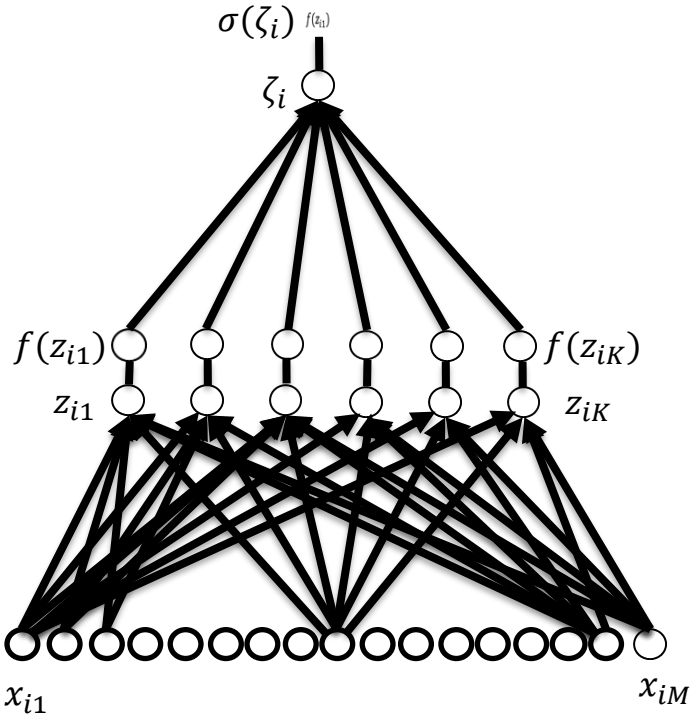


$f(\cdot)$ is a <u>nonlinear</u> function

Extended Logistic Regression



Extended Logistic Regression

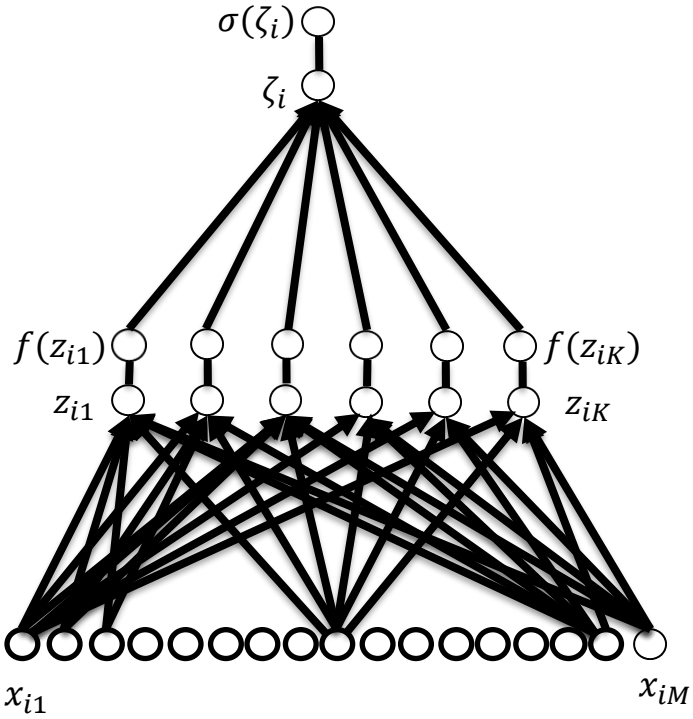


$$z_{i1} = b_1 + x_i \cdot w_1$$
$$z_{i2} = b_2 + x_i \cdot w_2$$
$$\vdots$$
$$z_{iK} = b_K + x_i \cdot w_K$$

K Features output from Layer-1

K Layer-1 filters

Extended Logistic Regression



Nonlinear function

Single Layer-2 Filter

$$\zeta_i = c_0 + f(z_i) \cdot c$$

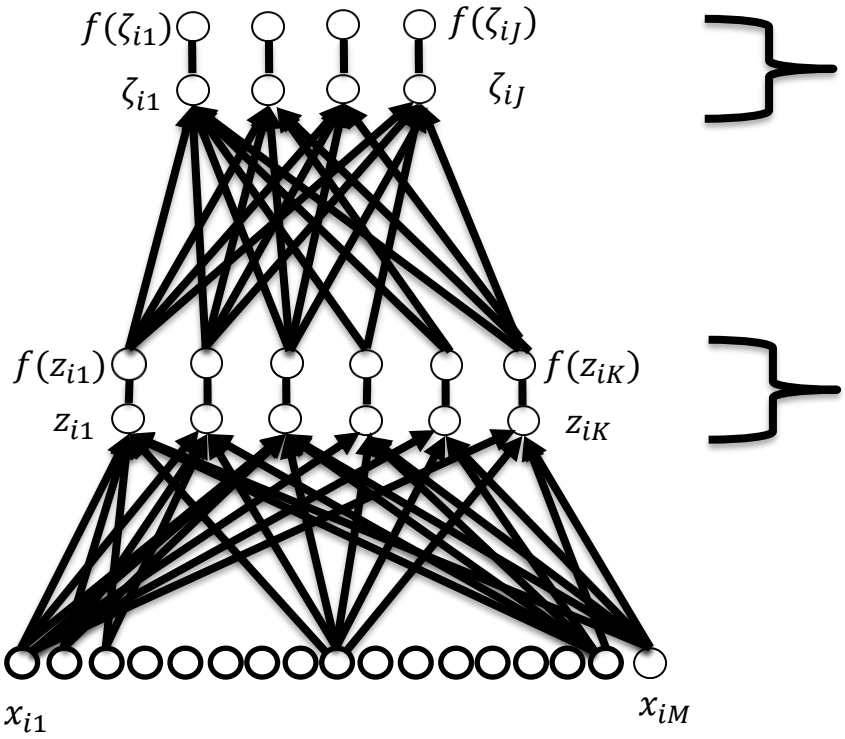
K-Dimensional Feature Vector Output from Layer-1

$$z_{i1} = b_1 + x_i \cdot w_1$$
$$z_{i2} = b_2 + x_i \cdot w_2$$
$$\vdots$$
$$z_{iK} = b_K + x_i \cdot w_K$$

K Features output from Layer-1

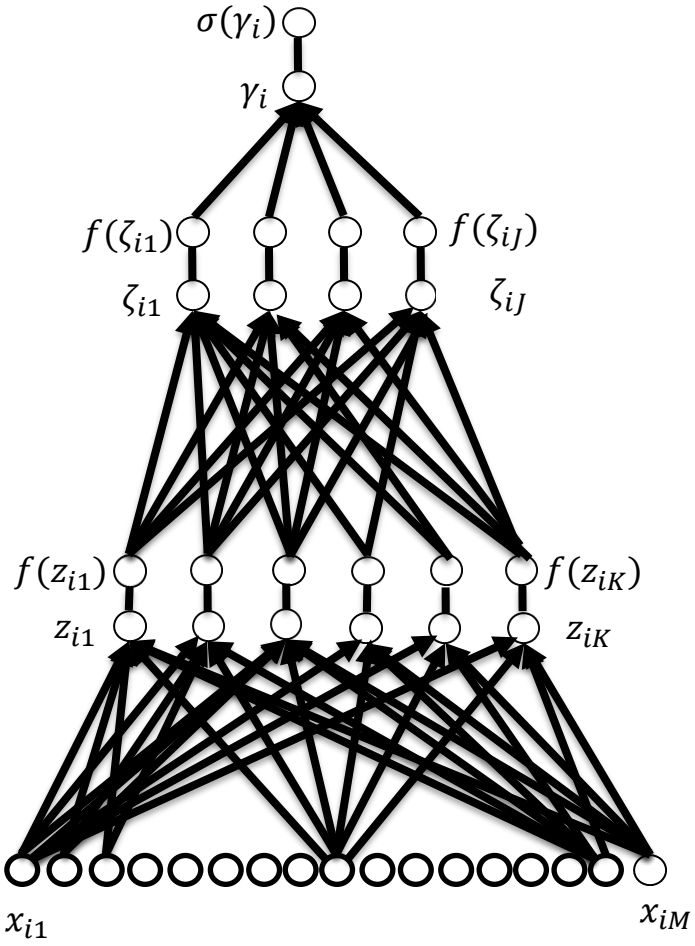
K Layer-1 filters

Going Deeper



Layer-2 latent features

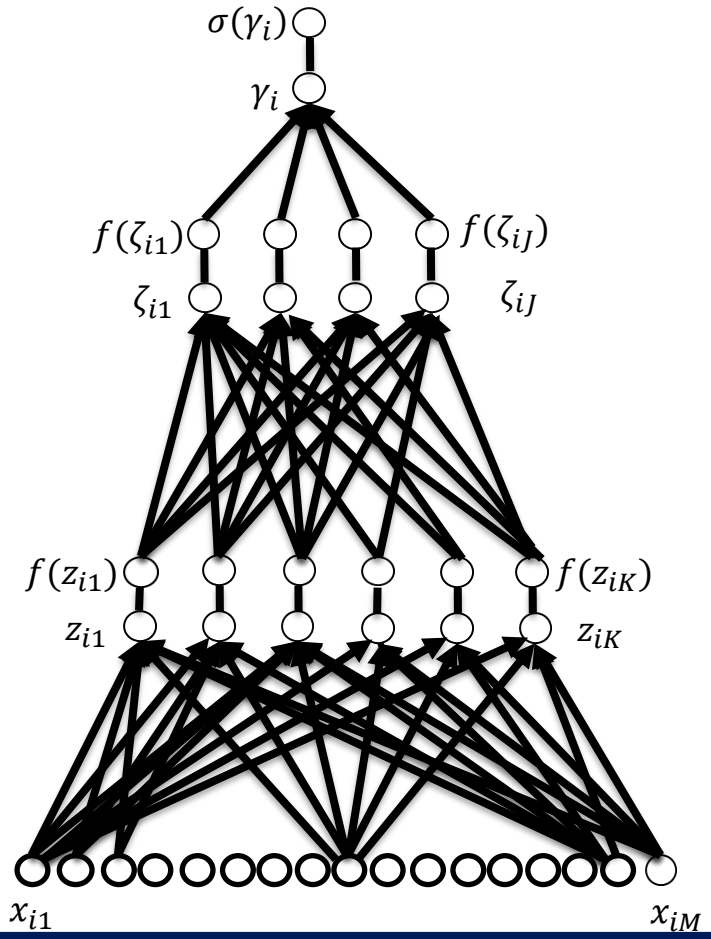
Layer-1 latent features



Probability of particular outcome

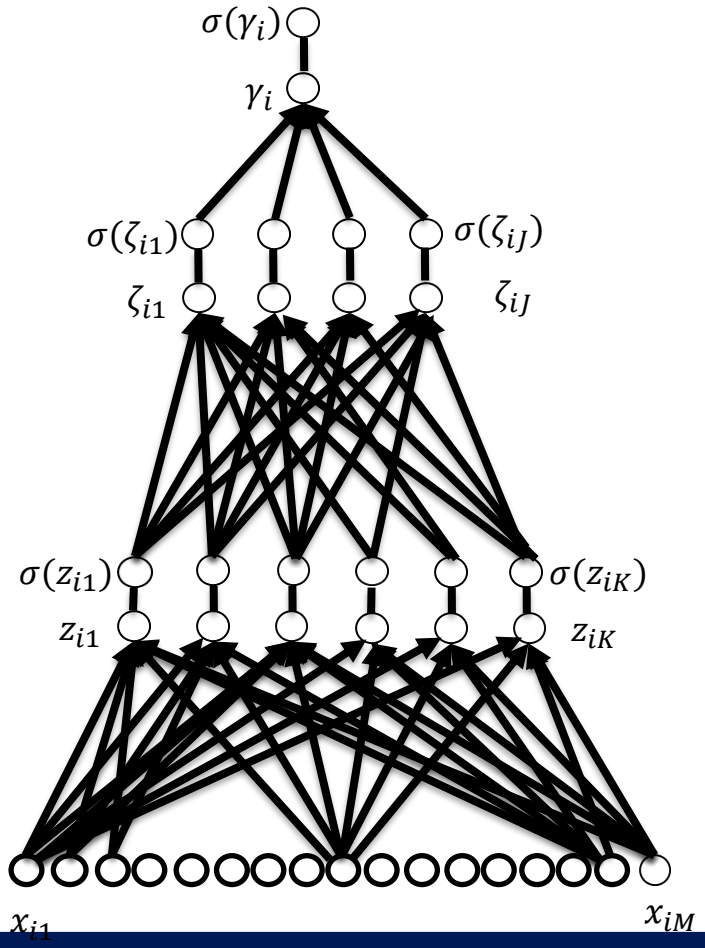
Layer-2 latent features

Layer-1 latent features



$$\left.\begin{aligned} z_{i1} &= b_{01} + x_i \cdot b_1 \\ z_{i2} &= b_{02} + x_i \cdot b_2 \\ &\vdots \\ z_{iK} &= b_{0K} + x_i \cdot b_K \end{aligned}\right\}$$

Features of the Data with Layer-1 Filters

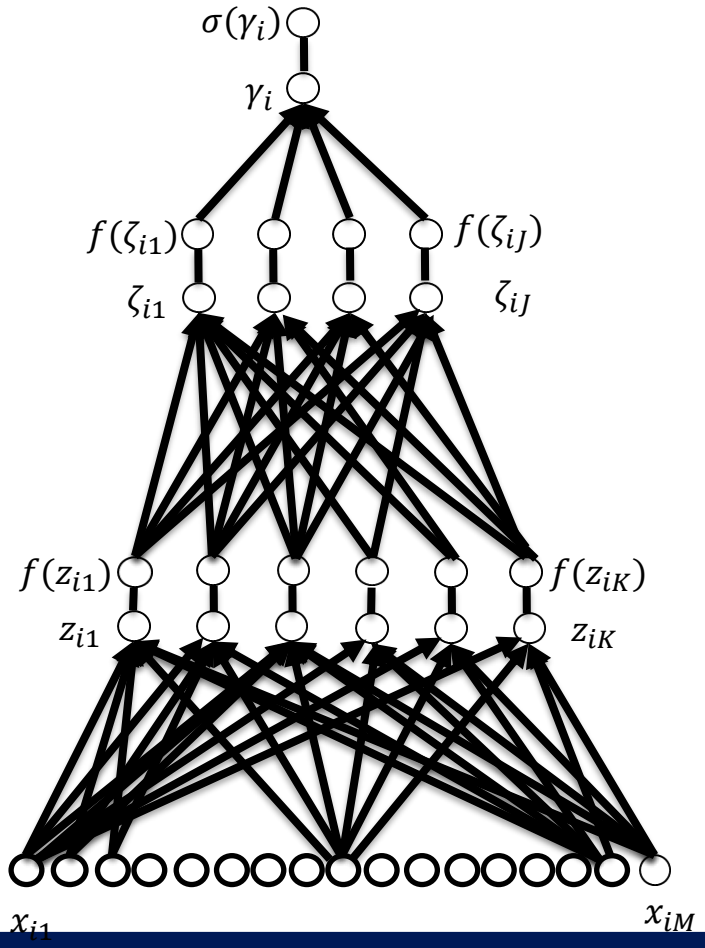


$$\left.\begin{aligned} \zeta_{i1} &= c_{01} + f(z_i) \cdot c_1 \\ \zeta_{i2} &= c_{02} + f(z_i) \cdot c_2 \\ &\vdots \\ \zeta_{iK} &= c_{0J} + f(z_i) \cdot c_J \end{aligned}\right\}$$

Features of the Layer-1 Features, with Layer-2 Filters

$$\left.\begin{aligned} z_{i1} &= b_{01} + x_i \cdot b_1 \\ z_{i2} &= b_{02} + x_i \cdot b_2 \\ &\vdots \\ z_{iK} &= b_{0K} + x_i \cdot b_K \end{aligned}\right\}$$

Features of the Data with Layer-1 Filters



$$\gamma_i = d_0 + f(\zeta_i) \cdot d$$

Logistic Regression Based on the Layer-2 Features

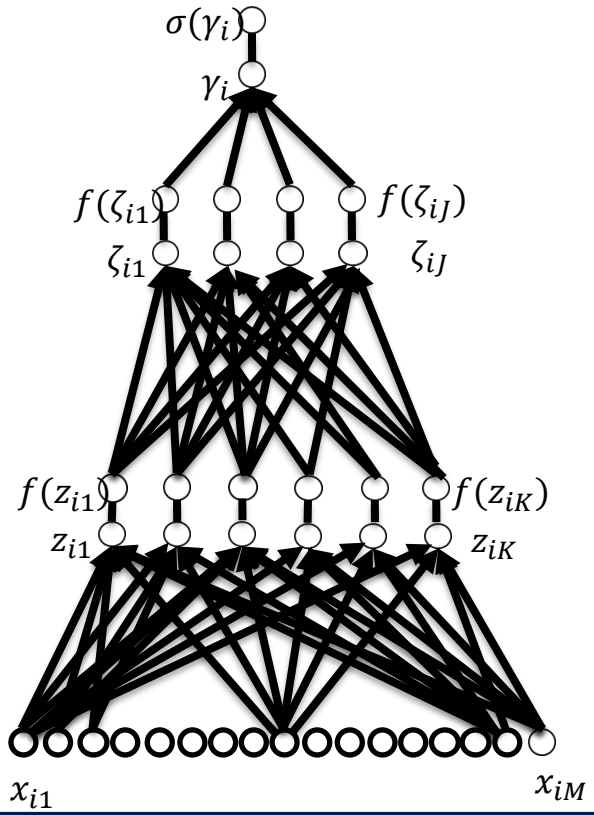
$$\zeta_{i1} = c_{01} + f(z_i) \cdot c_1$$
$$\zeta_{i2} = c_{02} + f(z_i) \cdot c_2$$
$$\vdots$$
$$\zeta_{iK} = c_{0J} + f(z_i) \cdot c_J$$

Features of the Layer-1 Features, with Layer-2 Filters

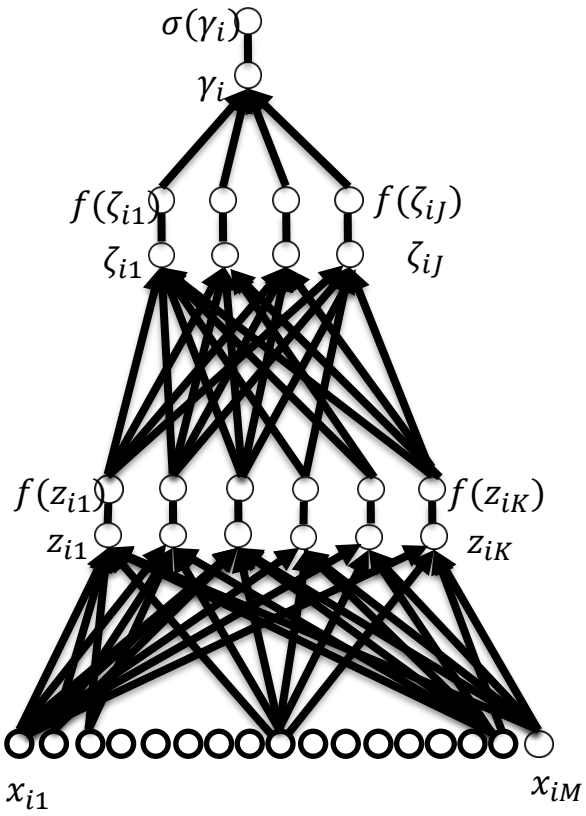
$$z_{i1} = b_{01} + x_i \cdot b_1$$
$$z_{i2} = b_{02} + x_i \cdot b_2$$
$$\vdots$$
$$z_{iK} = b_{0K} + x_i \cdot b_K$$

Features of the Data with Layer-1 Filters

Multilayer Perceptron: Neural Network Deep Learning



Multilayer Perceptron: Neural Network Deep Learning

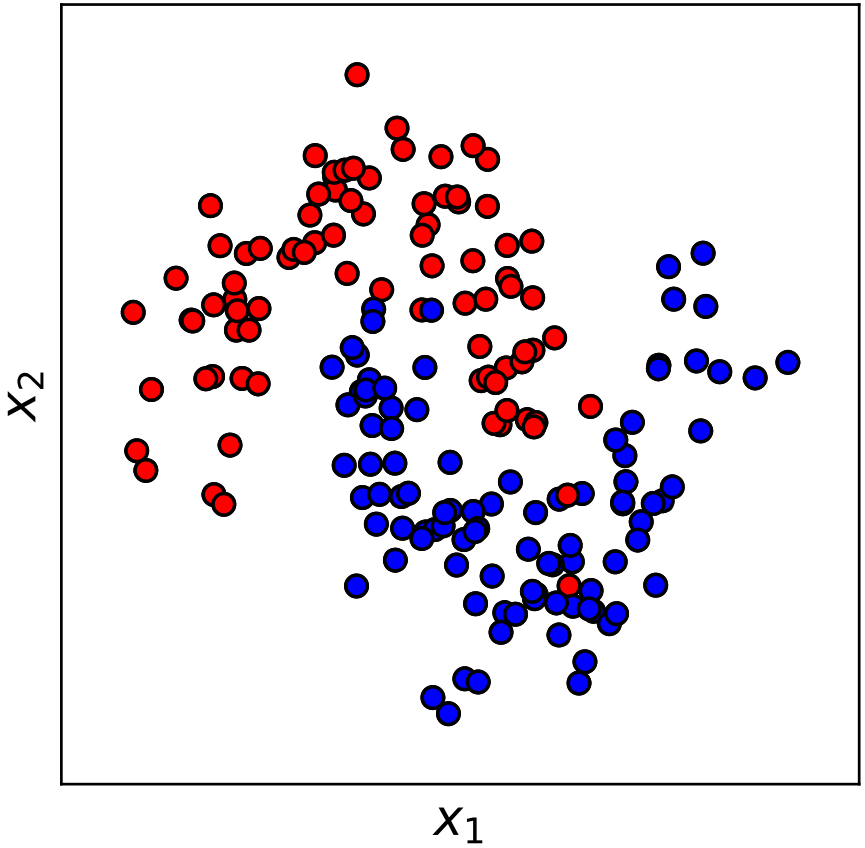


In modern NNs, based on our earlier discussions, learned features are typically normalized at each layer

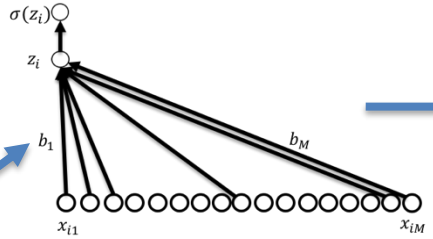
Input data also normalized

Can learn non-linear classifications

By learning multiple layers and transformation, it is possible to have a non-linear classifier capable of more accurately capturing the data.

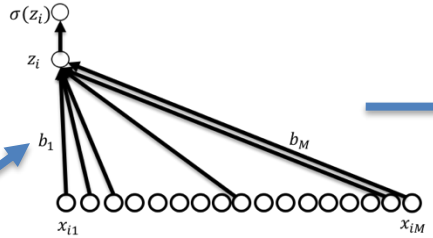


Does this work with MNIST?

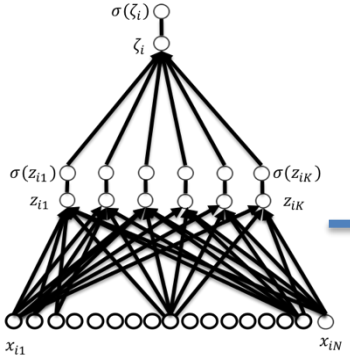


~91% Accurate

Does this work with MNIST?



~91% Accurate



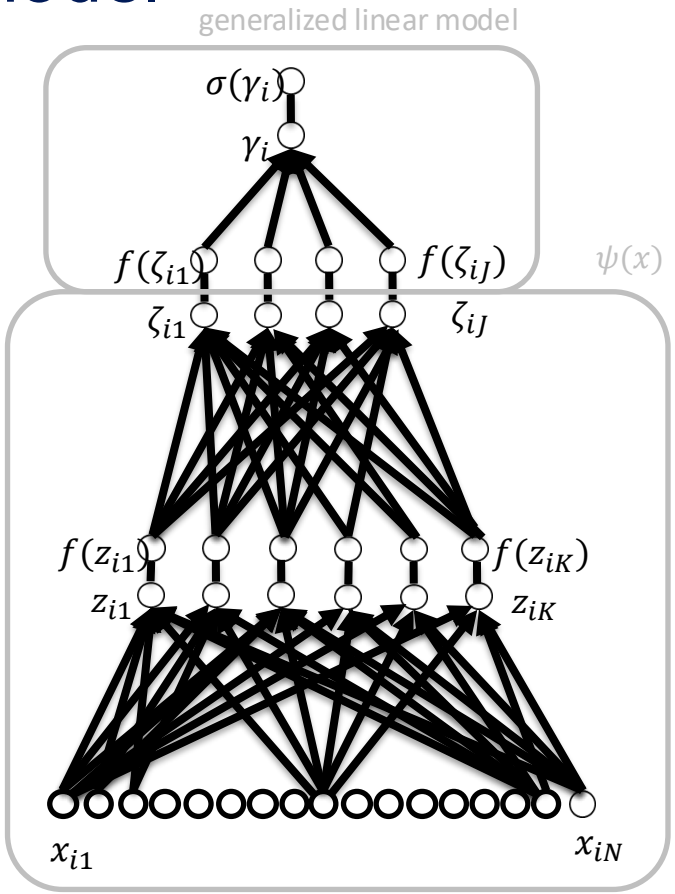
~96% Accurate

Perspective on this Model

- With logistic regression, or similar linear or generalized linear model, it is assumed data well analyzed along a line through the origin

- With an MLP, data x is transformed by a series of affine transformations followed by pointwise nonlinearities

- Critically important: At the output layer of the MLP, there is a generalized linear model, in the <u>feature space</u>

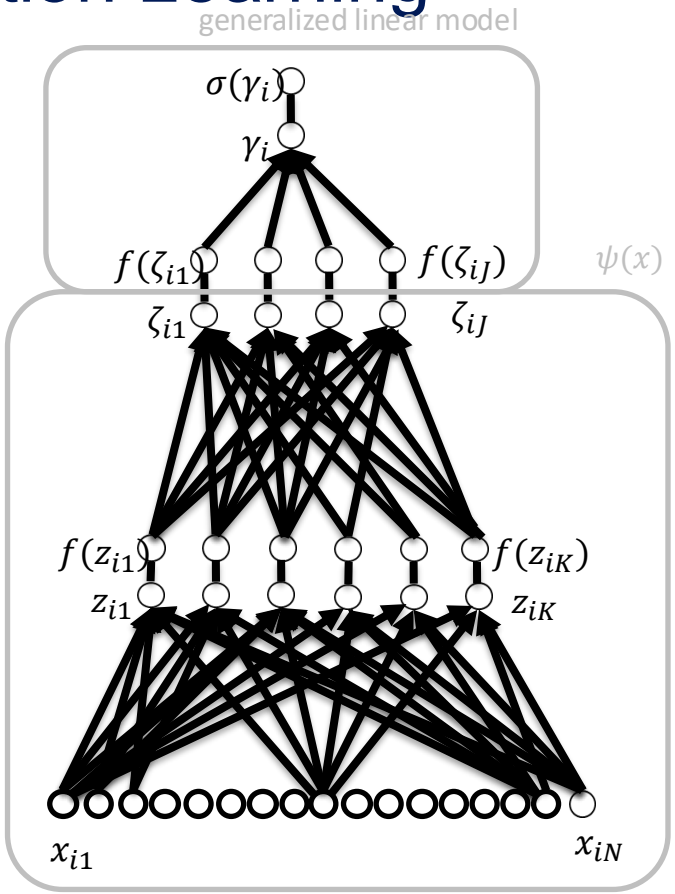


Deep Learning = Representation Learning

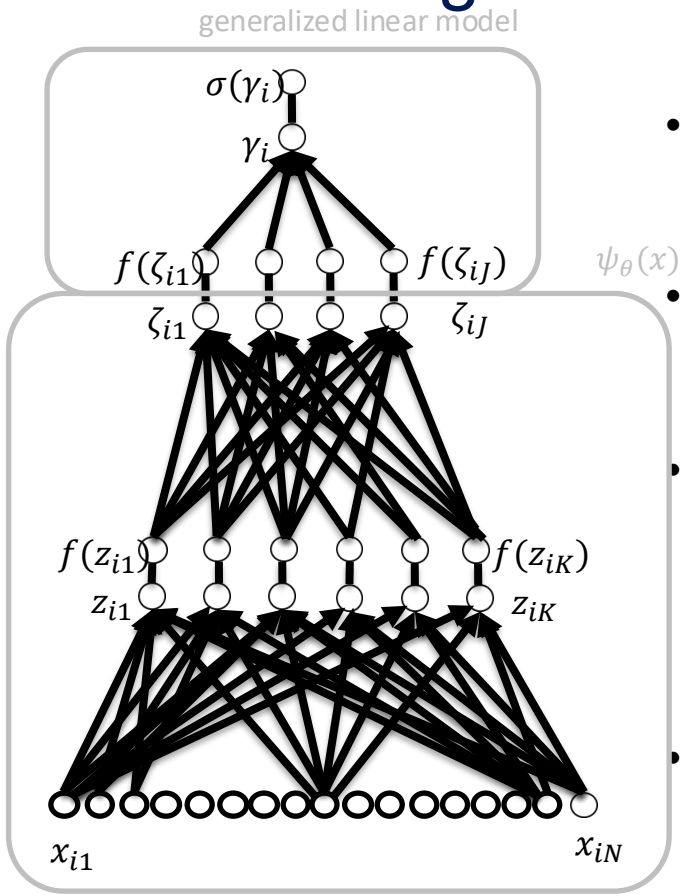
- We may see deep learning, manifested by multiple NN layers, as learning a nonlinear transformation of the data such that a linear model in the final representation is valid

- This is also called *representation learning*

- We almost always have a generalized linear model at the output layer, which is why understanding linear models well, is so important

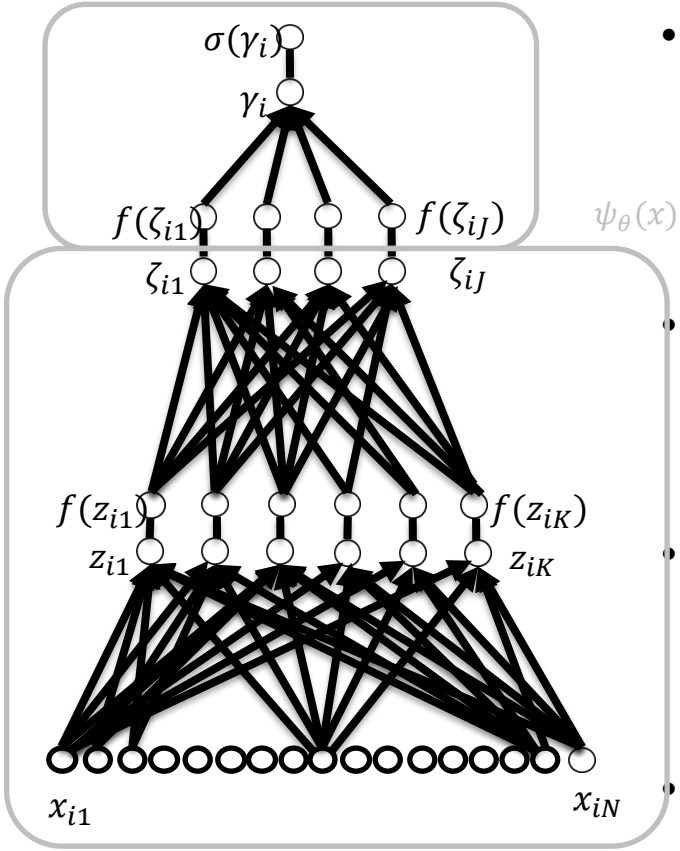


Thinking More About Representation Learning



- The input data $x \in \mathbb{R}^N$ and features $\psi_\theta(x) \in \mathbb{R}^J$
- In general, $N \neq J$ and possible $J > N$
- We look at this as a function $\psi_\theta(x)$ where θ are the NN model parameters (weights and biases at each layer)
- Function $\psi_\theta(x)$ represents the nonlinear portion of the NN

Thinking More About Representation Learning

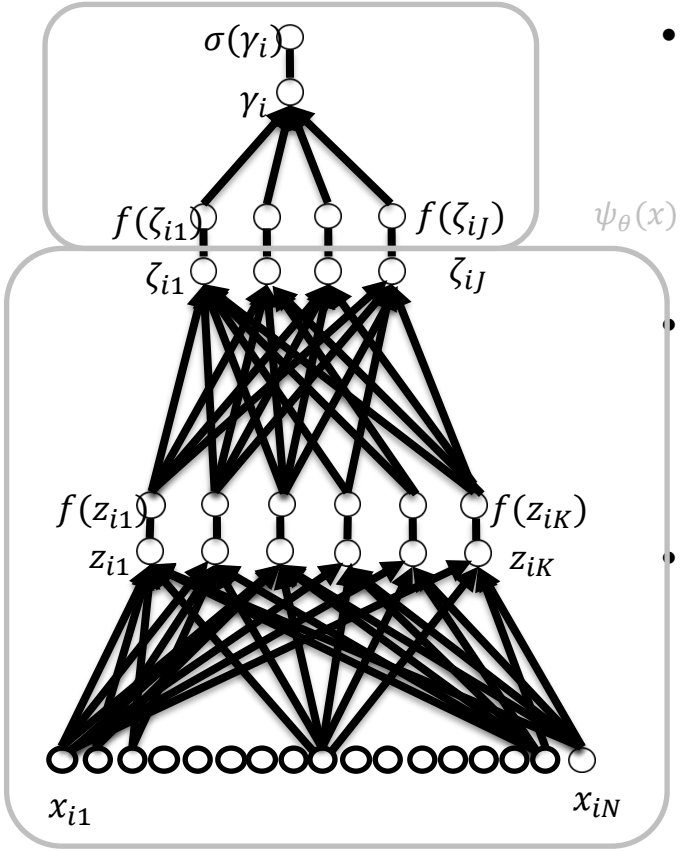


- The output of linear layer, before nonlinear $\sigma(\cdot)$, is

$$\gamma = w \cdot \psi_\theta(x) + b$$

- Vector $w \in \mathbb{R}^J$ resides in feature space
- Because of the properties of the inner product, w must reside within a linear subspace spanned by $\{\psi_\theta(x_i)\}_{i=1,M}$
- Assume that the training data composed of $\{(x_i, y_i)\}_{i=1,M}$

Thinking More About Representation Learning



- The output of linear layer, before nonlinear $\sigma(\cdot)$, is

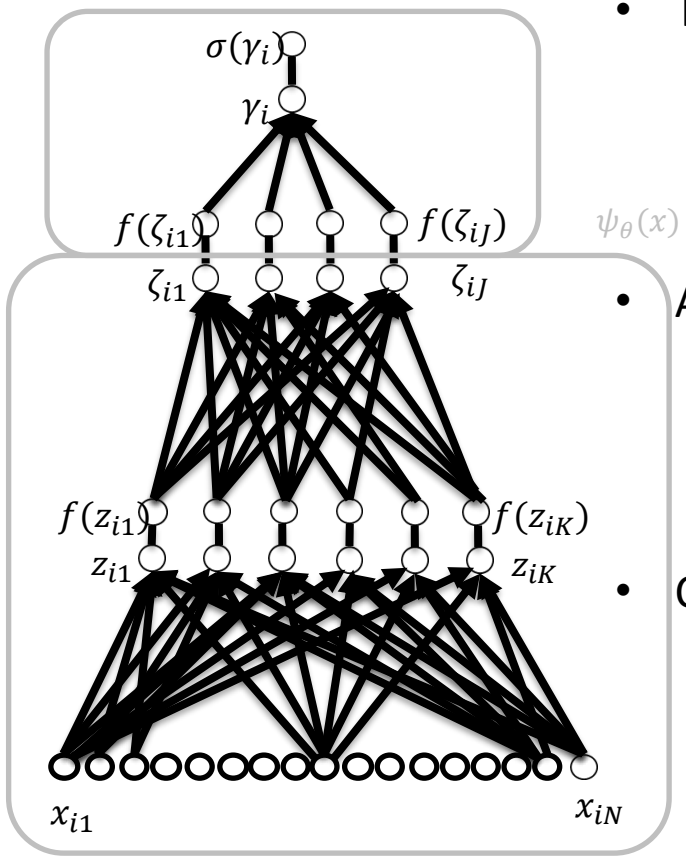
$$\gamma = w \cdot \psi_\theta(x) + b$$

- Assume that the training data composed of $\{(x_i, y_i)\}_{i=1,M}$

- This means we anticipate that w may be expressed as

$$w = \sum_{i=1}^{M} a_i \psi_\theta(x_i)$$

Kernel Learning



- The output of linear layer, before nonlinear $\sigma(\cdot)$, is

$$\gamma = w \cdot \psi_\theta(x) + b$$

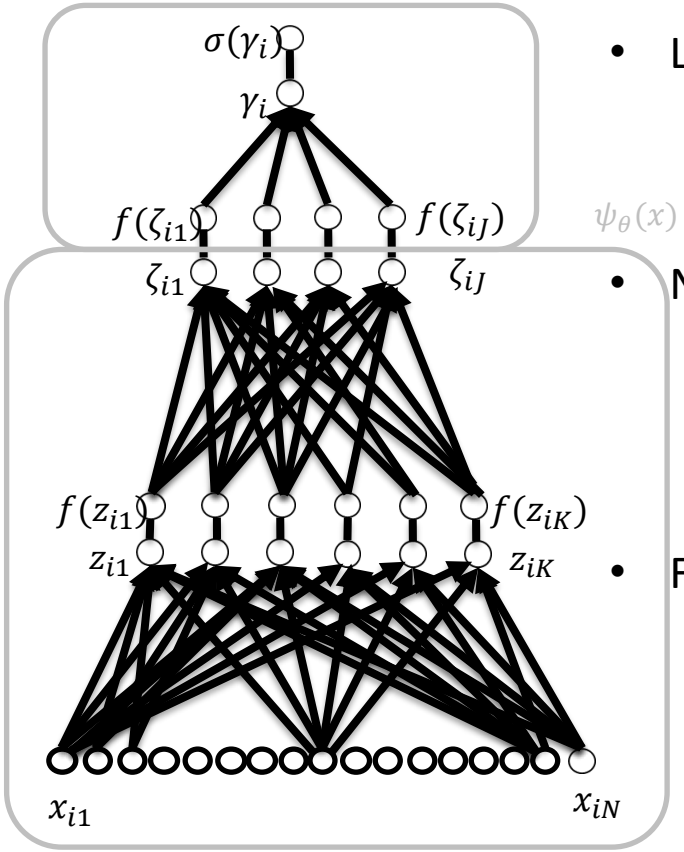
- Assume that the training data composed of $\{(x_i, y_i)\}_{i=1,M}$

$$w = \sum_{i=1}^{M} a_i \psi_\theta(x_i)$$

- Consequently:

$$\gamma = \sum_{i=1}^{M} a_i \psi_\theta(x_i) \cdot \psi_\theta(x) + b = \sum_{i=1}^{M} a_i k_\theta(x, x_i) + b$$

Putting it All Together



- Logistic regression: Generalized linear model in data space:

$$\sigma(x \cdot w + b)$$

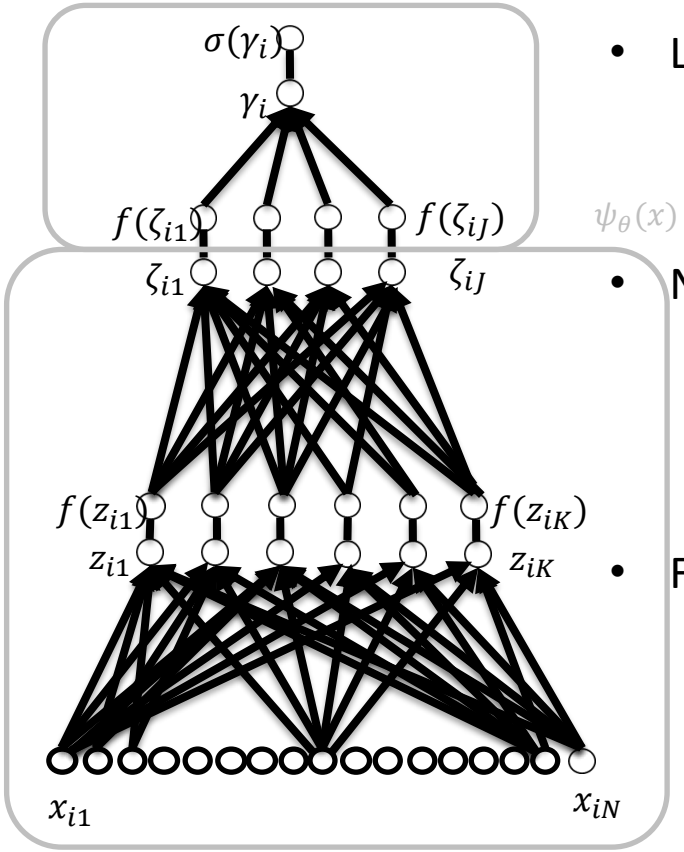
- Neural network: Generalized linear model in feature space:

$$\sigma[\psi_\theta(x) \cdot w + b]$$

- From perspective of kernel learning, the latter is

$$\sigma\{\sum_{i=1}^{M} a_i k_\theta(x, x_i) + b\}$$

Putting it All Together



- Logistic regression: Generalized linear model in data space:

$$\sigma(x \cdot w + b)$$

Linear from the perspective of x

- Neural network: Generalized linear model in feature space

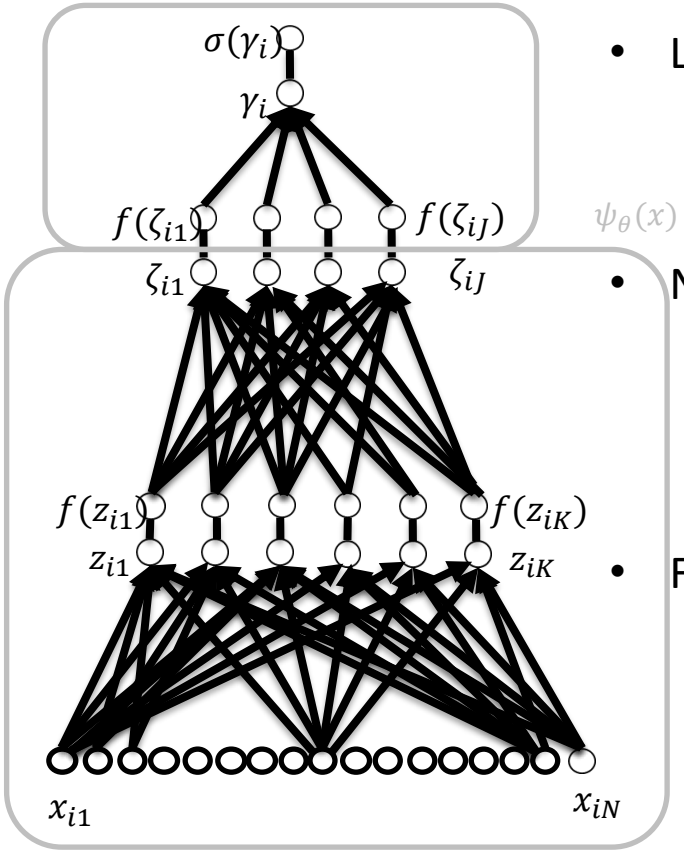
$$\sigma[\psi_\theta(x) \cdot w + b]$$

- From perspective of kernel learning, the latter is

$$\sigma\{\sum_{i=1}^{M} a_i k_\theta(x, x_i) + b\}$$

Nonlinear from the perspective of x

Putting it All Together



- Logistic regression: Generalized linear model in data space:

$$\sigma(x \cdot w + b)$$

Linear from the perspective of x

- Neural network: Generalized linear model in feature space

$$\sigma[\psi_\theta(x) \cdot w + b]$$

Linear in feature space $\psi_\theta(x)$

- From perspective of kernel learning, the latter is

$$\sigma\{\sum_{i=1}^{M} a_i k_\theta(x, x_i) + b\}$$

Nonlinear from the perspective of x

Historical Footnote: Kernel Machines

- Until relatively recently, there was not sufficient data and computation to reliably <u>learn</u> kernels like $k_\theta(x, x_i) = \psi_\theta(x) \cdot \psi_\theta(x_i)$

- Therefore, in practice kernels were "handcrafted", with a few parameters, *e.g.*, the radial basis function

$$k_\lambda(x, x') = \exp(-\lambda \|x - x'\|_2)$$

- Much smaller number of parameters to learn, here just one, λ

$$\sigma\{\sum_{i=1}^{M} a_i k_\lambda(x, x_i) + b\}$$

Historical Footnote: Kernel Machines

- Until relatively recently, there was not sufficient data and computation to reliably <u>learn</u> kernels like $k(x, x_i) = \psi_\theta(x) \cdot \psi_\theta(x_i)$

- Therefore, in practice kernels were "handcrafted", with a few parameters, e.g., the radial basis function

$$k_\lambda(x, x') = \exp(-\lambda \|x - x'\|_2)$$

- Much smaller number of parameters to learn, here just one, λ

$$\sigma\{\sum_{i=1}^{M} a_i k_\lambda(x, x_i) + b\}$$

Widely used example: Support Vector Machine (SVM), in which the set of weights $\{a_i\}_{i=1,M}$ is *sparse*

Transformers as Learned Kernel Machines

- Very difficult to “handcraft” kernels for sophisticated data, like images, sound and natural language (*e.g.*, Euclidean distance is not a good metric of closeness)

- Deep learning and NNs may be framed as kernel learning for general data

- The Transformer network, which is the focus of this class, is a very powerful form of nonlinear feature learning, particularly for natural language, images, and related data

- The output layer of a Transformer is linear, or generalized linear, and therefore like virtually all NNs, it may be framed as representation learning, or equivalently kernel learning